Universidad ORT Uruguay
Facultad de Ingeniería



Entregable 1 - Ingeniería de Software Ágil 2

Estudiantes: Iván Casales (257790), Francisco Gandioli (290071), Matías Trujillo (272561)
Profesores: Santiago Gonzalez, Juan Irabedra
Año de entrega: 2025

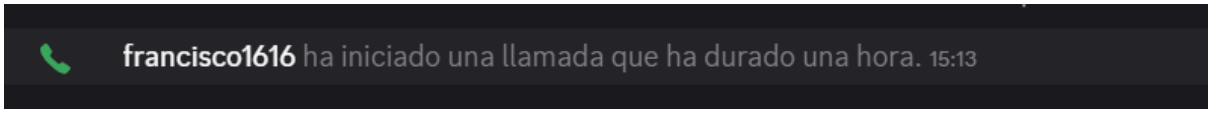
Parte 1

1. Registro de Actividades

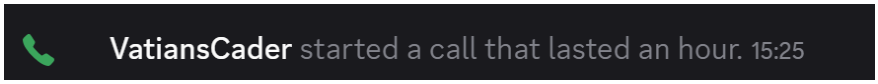
Registro de actividades realizadas (fecha, actividad, horas, integrante), cada actividad debe tener al menos un commit en el repositorio realizado en la fecha reportada:

Nota: Como en está primera entrega casi no hubo commits, ya que se basó en realizar documentación y organizar el repo, no realizamos un seguimiento de la parte de “cada actividad debe tener al menos un commit en el repositorio realizado en la fecha reportada”.

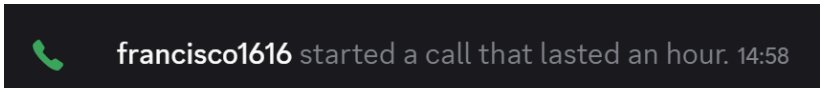
12/9/25

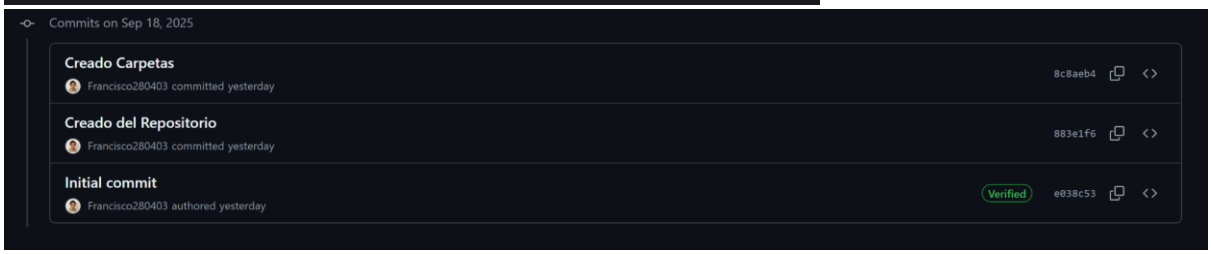


15/9/25

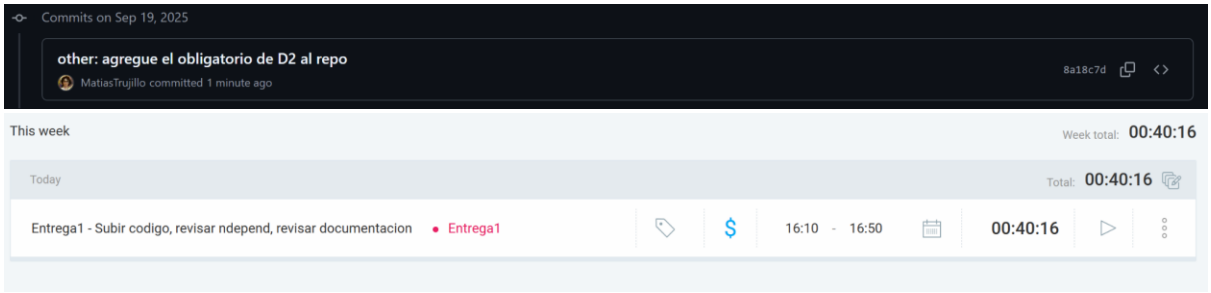


18/9/25





19/9/25



20/9/25

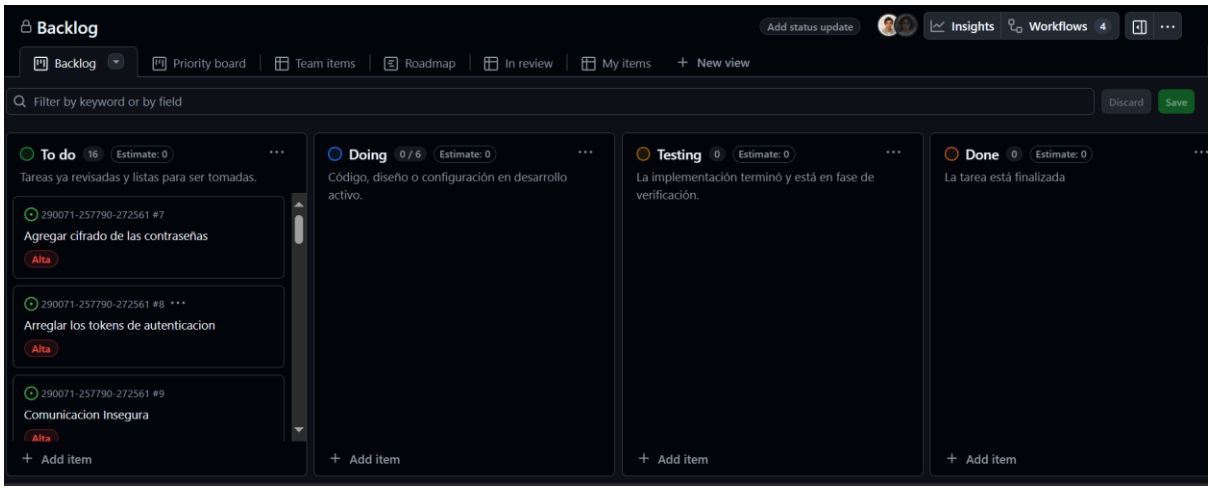


2. Evidencia de Resultados

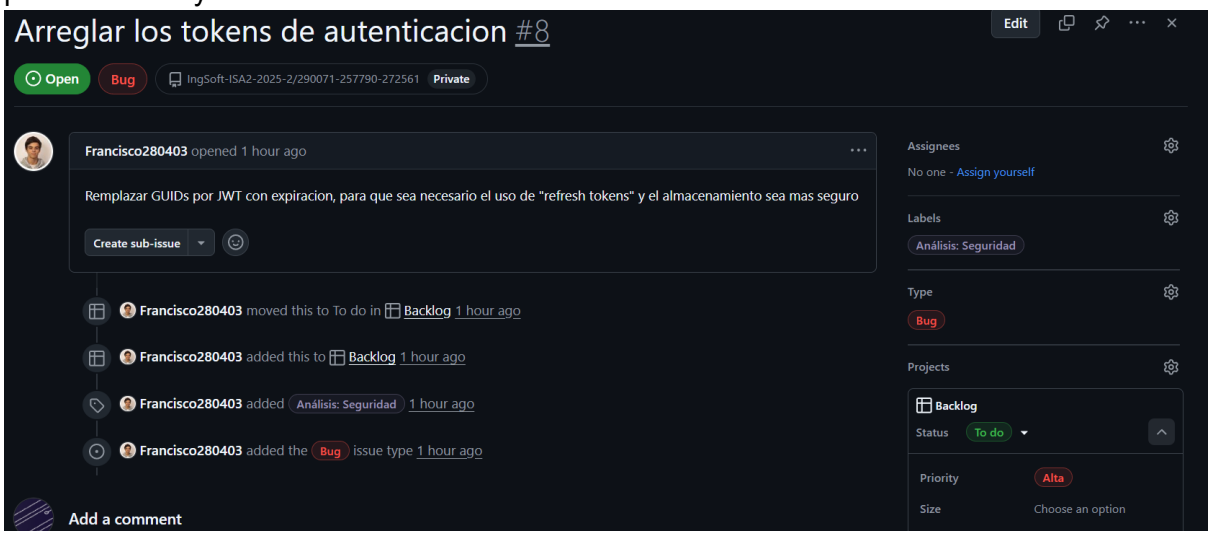
Evidencia de resultados de la entrega (detallados en la rúbrica de evaluación)
En el repo, en /Informes/Primera_entrega.md se encuentran todas las secciones de la rúbrica de forma detallada y clasificada por títulos con el mismo órden.

Link al repositorio creado en la organización:
[IngSoft-ISA2-2025-2/290071-257790-272561]

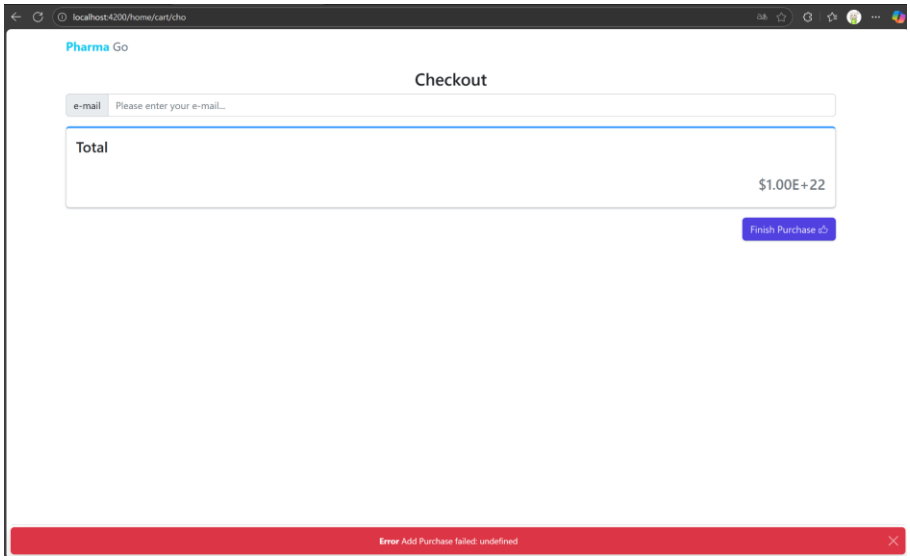
También, dentro de la sección Projects del repositorio se encuentra el tablero con las definiciones realizadas en el informe markdown. A continuación un captura de pantalla del tablero con sus columnas:



A continuación una captura de pantalla de un ticket (issue), donde se puede visualizar el título, la descripción, y a la derecha el tipo de análisis: Seguridad, El tipo de ticket: Bug, la prioridad: Alta y el estado: To do.



A continuación podemos ver la captura de pantalla del momento en que encontramos un error en la aplicación, el cual fue documentado en el archivo markdown, y luego convertido a Issue en el tablero del proyecto de Github.



3. Referencias y Documentación

Link a la documentación MarkDown donde se encuentra el informe realizado en base a la rúbrica:
[290071-257790-272561/Informes/primera_entrega.md at main · IngSoft-ISA2-2025-2/290071-257790-272561]

4. Mejoras y Revisión con el PO (Review)

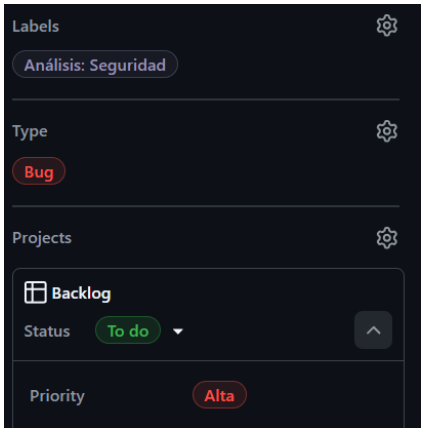
Mejoras en la calidad y evidencia de revisión con el PO (uno de los integrantes debe tomar el rol en cada etapa).
Durante ésta etapa, el avance en sí fue de análisis del obligatorio de Diseño 2, y generación de documentación y creación de tareas en base a los resultados obtenidos.

Análisis del PO:

- El equipo presentó un repositorio organizado, tablero Kanban operativo y Definitions of Ready/Done bien documentadas.
- Validaciones: Se confirmó la correcta estructura del repositorio, la configuración del tablero con columnas y WIP definido, y un análisis inicial de funcionalidad, mantenibilidad y seguridad.

Cosas que se mejoraron:

- En un inicio, la prioridad de las tareas se asignó como una label. Luego para mayor consistencia se utilizó la tabla ya definida “Priority” para determinar la importancia.



- El PO analizó las prioridades y se corrigieron algunas, ya que en algunos casos teníamos tareas de prioridad alta que se bajaron a medio, y algunas de media las bajamos a baja.

Nota: Si bien, generalmente la Review se realiza una vez finalizado todo, y las cosas a mejorar suelen llevarse a la siguiente iteración, en nuestro caso mejoramos las observaciones del PO luego de la misma.

5. Retrospectiva y Acciones de Mejora (Retrospective)

Retrospectiva y acciones de mejora

Aspectos positivos:

- Si bien los integrantes del equipo tienen distintos horarios debido a trabajo, aún así se pudieron organizar correctamente para realizar las llamadas y lograr avances en la entrega.
- La comunicación fue muy efectiva, hablando por whatsapp para organizarse y realizando llamadas por discord para trabajar de forma sincrónica.
- La utilización de herramientas de trabajo sincrónico como Google Docs facilitó la coordinación y el trabajo simultáneo.

Aspectos negativos:

- Como durante ésta primera entrega se basó principalmente en realizar documentación, los roles no fueron muy distinguibles, realizando tareas muy similares. Incluso, si bien uno de los 3 integrantes fue el PO y realizó el principal aporte en la Review, los otros dos también lo ayudaron y lo complementaron para analizar los incrementos.

Cosas a mejorar:

- Mejorar la organización de las tareas entre los integrantes de los equipos. Ya que en este caso no pusimos responsables previamente a los pendientes, sino que lo fuimos definiendo mientras trabajabamos juntos durante las llamadas
- Tener días fijos de la semana para juntarse a trabajar, para mantener una mejor consistencia en los resultados.
- Intentar que los integrantes del equipo actúen más similar a los roles asociados a la entrega.

Parte 2

1. Definición de marco de gestión basado en Kanban

Prácticas principales

- Swarming: reunión de emergencia cuando un integrante se encuentra bloqueado, para resolver el impedimento de forma colaborativa.
- Límite de WIP: se define un máximo de tarjetas en las columnas Doing y Testing para evitar trabajo en paralelo excesivo.
- Definition of Ready (DoR): criterios que una tarea debe cumplir para ingresar al tablero (claridad en requisitos, criterios de aceptación y dependencias resueltas).
- Definition of Done (DoD): criterios para considerar una tarea finalizada (código revisado, pruebas automáticas en verde, cobertura mayor a 90 %, documentación actualizada).
- Mejora continua: revisión periódica del flujo de trabajo en las retrospectivas, ajustando límites de WIP, columnas y métricas según aprendizajes.

Definition of Ready (DoR):
Título: Nombre breve y representativo de la tarea/issue.
Descripción: Detalle claro del objetivo, alcance y contexto.
Tipo: Bug, Feature, Task
Tipo de análisis: Mantenibilidad - Funcionalidad - Seguridad (pueden seleccionarse varios si aplica).
Prioridad: Alta - Media - Baja, según impacto y urgencia.
Estado: To Do - Doing - Testing - Done, depende de qué columna del tablero se encuentre.

Definition of Done:
Una tarea o historia de usuario se considera terminada solo si cumple todos los siguientes criterios:

- Código completo y revisado
    - Implementación finalizada, revisada en Pull Request por al menos un integrante distinto del autor.
    - Estilo y estándares validados por linters y formateadores automáticos.
- Pruebas y calidad
    - Cobertura mínima del 90 % en pruebas unitarias y de integración.
    - Todas las pruebas automáticas pasan en GitHub Actions sin errores.
    - Se corrigen defectos encontrados en la fase de Testing.
- Integración continua
    - Build exitoso en la pipeline de CI/CD.
    - Merge a la rama main realizado solo tras aprobar revisiones y pasar validaciones automáticas.
- Documentación
    - Código comentado cuando es necesario.

        - Documentación técnica o de usuario actualizada (README, wiki, etc.) según corresponda.
- Trazabilidad y registro
    - Tarjeta de Kanban actualizada a “Done” con el enlace al commit/PR correspondiente.
    - Registro de horas en Clockify para la actividad.

Medición

- Clockify para registrar y analizar el tiempo dedicado a cada tarea y obtener métricas de productividad y capacidad.

Tablero Kanban

Columnas: To Do, Doing, Testing y Done:

To Do

- Tareas ya revisadas y con Definition of Ready completo, listas para ser tomadas.
- Aún no tienen trabajo iniciado.

Doing

- Trabajo en curso por uno o más integrantes.
- Código, diseño o configuración en desarrollo activo.
- Se registra el tiempo en Clockify y se actualiza el avance en la tarjeta.
- Mantener límites de WIP para evitar sobrecarga.
    - **WIP actual: 6**
        - Elegimos 6 porque 3 nos pareció poco, y queríamos tener un múltiple de 3 debido a que somos 3 integrantes.

Testing

- La implementación terminó y está en fase de verificación.
- Incluye pruebas unitarias, integración y validaciones de criterios de aceptación.
- Debe cumplir con una cobertura mayor a 90 % y pasar GitHub Actions antes de avanzar.
- Se corrigen defectos encontrados en esta etapa.

Done

- La tarea está finalizada y cumple con la Definition of Done.
- Código mergeado a main sin fallas.
- Documentación actualizada y revisada por al menos un integrante.
- Listo para entrega o despliegue en producción según corresponda.

Roles

Roles: developers, product owner, tester, Scrum master

Todos los miembros actúan como Desarrollador y Tester.

- Ivan: Developer, Tester
- Matías: Dev, Tester, Product Owner
- Francisco: Dev, Tester, Scrum Master

(Los roles de Product Owner y Scrum Master rotarán en entregas futuras)

Ceremonias

- **Weekly:** Se tomó esta decisión en base a SCRUM y a como nos queda mejor para trabajar en la semana. Esto es principalmente debido a que integrantes del equipo trabajan, por lo que los horarios de reunión está más limitado a un dia entre semana y los finde de semana
- **Review:** Reunión al finalizar la iteración en la que el equipo presenta el incremento del producto al Product Owner y demás interesados.
    - Su objetivo es validar las funcionalidades, recopilar feedback y decidir ajustes o nuevas prioridades en el backlog para el próximo ciclo.
- **Retrospective**: Sesión interna del equipo que se realiza después de la review para reflexionar sobre el proceso de trabajo.
    - Se analizan prácticas, comunicación y herramientas, identificando aciertos, problemas y acciones concretas de mejora para la siguiente iteración.

2. Definición de un primer proceso de ingeniería

A continuación vamos a definir la forma de trabajo a realizar

1. Control de versiones y repositorio
    - Plataforma: GitHub.
    - Estrategia de branching: trunk-based con ramas cortas para cada feature o bug:
        - Ejemplo: feature/*
    - Nomenclatura de commits: “tipo: descripción breve”.
        - Tipos de ejemplo para utilizar: feat, fix, docs, refactor, test.
        - Ejemplo: feat:función agregada
    - Flujo de trabajo de commits
        - Crear nueva branch a partir de main.
        - Desarrollar usando las metodologías acordadas.
        - Hacer commit siguiendo el estándar de mensajes.
        - Hacer push de la branch al repositorio remoto.
        - Abrir Pull Request a main, asegurando que la cobertura de pruebas sea mayor a 90 % y que el código sea revisado por al menos un integrante.
        - Git merge a main solo si pasan todas las GitHub Actions.
        - Repetir este ciclo.

2. Prácticas de desarrollo
    - Revisión de código obligatoria antes de cada merge (indicando los reviewers en el pull request).
    - Linters y formateadores automáticos.
    - Pair programming opcional para funcionalidades críticas.

3. Testing
    - Cobertura mínima de 90 % para permitir merges.
    - Ejecución automática de tests en cada push y pull request mediante GitHub Actions.

4. Integración continua (CI)
   - Pipeline de build + linters + pruebas. En un futuro se puede revisar y mejorar.
   - Merge a main bloqueado si falla cualquier paso.

5. Mejora continua
   - Revisión final por el PO para validación y futuras mejoras en los incrementos.
   - Retroalimentación en cada retrospectiva.
   - Ajustes documentados en los informes de avance.

3. Configuración del repositorio en GitHub

Organización repo:
- ObligD2: Obligatorio a analizar y mejorar.
- docs: Documentación nuestra (ejemplo: letra del obl de ISA2, Libro, etc)
- .gitignore
- Informes: Archivos .md con las entregas
- README.md: Readme general

Estrategia inicial de branching: Actualmente, como solo copiamos el reporte, lo organizamos y subimos documentación, únicamente utilizamos la rama main por simplicidad. Sin embargo, como se aclaró más arriba, en un futuro más a utilizar metodologías de branching como trunk based.
Estándares de nomenclatura: Más arriba se definieron los estándares de nomenclatura para los commits.

4. Análisis de deuda técnica y gestión de la calidad

Definición del modelo de calidad

Usando como referencia el modelo de calidad ISO25010, seleccionamos únicamente las columnas: function suitability (que las cosas funcionen como se espera) y mantenibilidad.

Function suitability

Elegimos function suitability porque nos importa que el sistema haga lo que tiene que hacer y cumpla con lo que piden los usuarios. Si no cubre bien las funciones, el resto no tiene sentido.

Mantenibilidad

Elegimos la mantenibilidad porque ya lo vimos en Diseño 1 y 2, y sabemos que es clave que el código sea fácil de entender, modificar y extender sin romper todo lo demás.

Seguridad

Elegimos la seguridad porque los integrantes del equipo se encuentran cursando la materia de seguridad informática. Si bien actualmente hemos visto principalmente contenido teórico, nos ha parecido interesante investigar por nuestra cuenta más a nivel técnico.

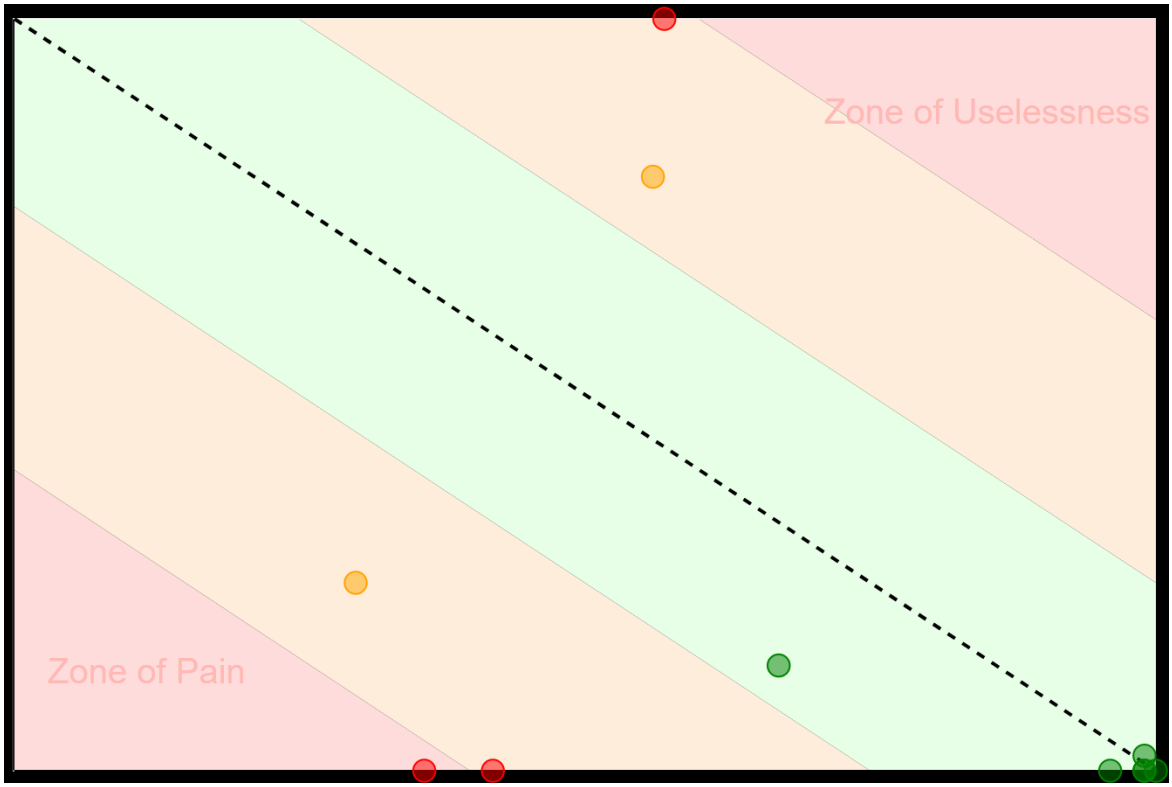
Análisis de deuda técnica

Function suitability

- **Falta de acceso cómodo a login/registro**: No hay una forma cómoda de ir a login o registro.Actualmente hay que escribir a mano el link para entrar en la página de login.
- **Ausencia de usuario administrador inicial**: Al principio la base de datos estaba vacía, por lo que no tenía usuario admin, lo cual es necesario, y la única forma de crearlo fue insertando directamente en la base de datos.
- **Búsqueda por similitud:** Cuando se buscan objetos, la búsqueda es por exactitud en lugar de similitud, lo que puede resultar incómodo para el usuario: Ejemplo si buscás “Objeto”, no se aparece “Objeto1”.
  Si bien no es en sí un error, y corresponde más a nivel de accesibilidad, consideramos corregirlo como una funcionalidad incompleta.
- **Error en tracking Code:** Si escribimos un tracking code inválido, la aplicación nos muestra un error, pero aún así lo agrega a la lista de tracking code en lugar de rechazarlo y no agregarlo.
- **Manejo de los mensajes de error del sistema:** El sistema del manejo de errores en algunos casos no muestra los errores reales, con mensajes genéricos. Por ejemplo, al ingresar un número mayor al integer max value sale el error: “Error Add Purchase failed: undefined”.

Mantenibilidad

- **Mejoras respecto abstracción-estabilidad y complejidad ciclomática**: Para el siguiente análisis utilizamos la herramienta NDepend, lo cual es una herramienta de análisis estático para proyectos .NET que ayuda a medir, visualizar y mejorar la calidad del código. Resultados:



  - Mejorar relación abstracción-estabilidad de los paquetes Domain, Exceptions y InstrumentationInterface
  - Reducir complejidad ciclomática de los paquetes WebApi, Domain y BusinessLogic.
  - Mejorar el índice de mantenibilidad del paquete DataAccess

- **Codificación de caracteres**: Los acentos y la letra ñ se muestran mal (ej. “ó” → “¦ü”) en carpeta de documentación.
  - Inconsistencia en nombres: Archivos con nombres desiguales dificultan la lectura, búsqueda y gestión en sistemas de control de versiones como Git.
  - Problemas de portabilidad: Estas inconsistencias generan errores en distintos sistemas operativos y complican herramientas de CI/CD y la colaboración internacional.

  - **Código repetido en manejo de errores**
    Unificar la lógica de manejo de errores en un servicio centralizado para evitar duplicación y mejorar la mantenibilidad.

  - **Falta de comentarios y documentación**
    Agregar documentación básica en métodos y clases para mejorar la comprensión futura.

  - **Nombres de métodos demasiado genéricos**
    Renombrar métodos como handleError o log a nombres más descriptivos.

Seguridad

Sin Cifrado de Contraseñas

- Contraseñas en texto plano: Las contraseñas se almacenan sin ningún tipo de protección.
- Ausencia de hashing
- Comparación insegura: Se realiza comparación directa de contraseñas (!user.Password.Equals(password)).

Tokens de Autenticación Inseguros
- Sin uso de JWT: El sistema utiliza GUIDs simples como tokens.
- Sin expiración: No se implementa TTL (Time To Live).
- Sin refresh tokens: No existe mecanismo de renovación de tokens.
- Almacenamiento inseguro: Los tokens se guardan en la base de datos sin encriptación.

Comunicación Insegura
- Falta de HTTPS obligatorio: No se exige conexión segura para las comunicaciones.
- Headers de autorización incorrectos: No se usa el prefijo Bearer.

Gestión de Sesiones Vulnerable
- Sesiones sin expiración automática: Las sesiones permanecen activas indefinidamente.

Falta de Validación de Entrada
- Riesgo de inyección: No se validan ni sanitizan los datos de entrada, lo que expone a ataques de inyección SQL u otros.

5. Definición de issues y clasificación

En base al tablero y a la Definition of Ready, realizamos la creación de los Issues en el proyecto del repositorio de Github y los asignamos en la columna “To do”

Entregable 2 - Ingeniería de Software Ágil 2

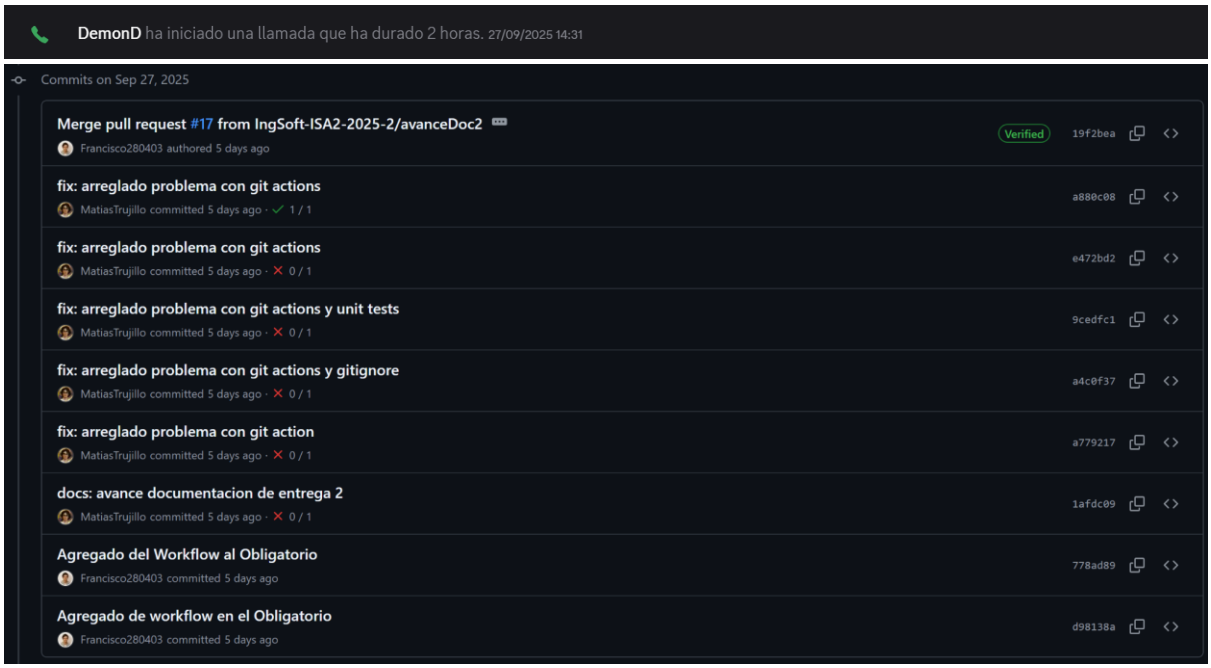
Estudiantes: Iván Casales (257790), Francisco Gandioli (290071), Matías Trujillo (272561)
Profesores: Santiago Gonzalez, Juan Irabedra
Año de entrega: 2025

Parte 1

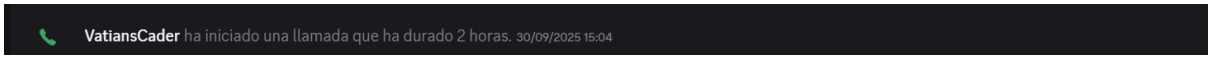
1. Registro de Actividades

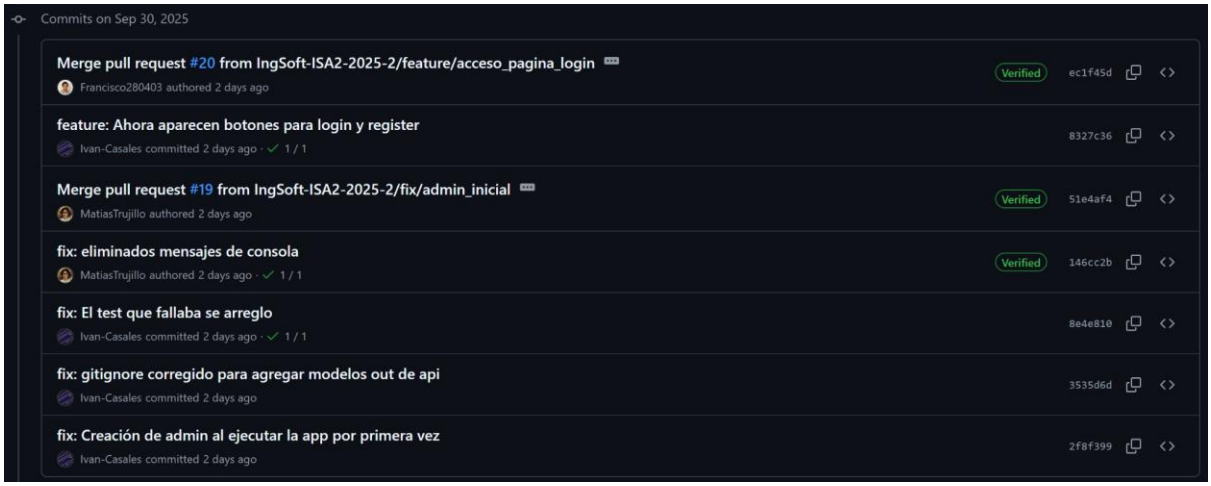
Registro de actividades realizadas (fecha, actividad, horas, integrante), cada actividad debe tener al menos un commit en el repositorio realizado en la fecha reportada:

27/9/25

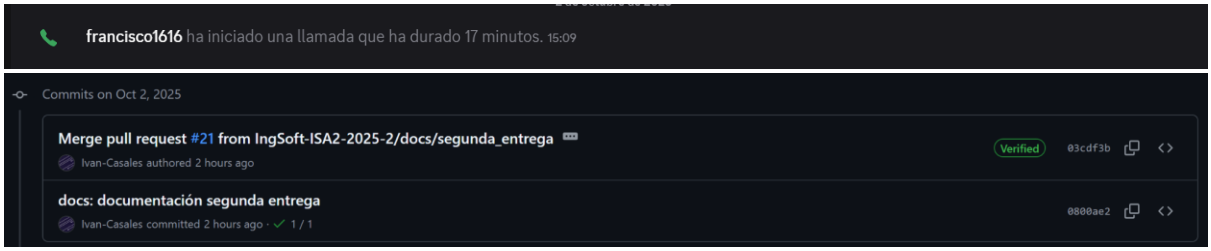


30/09/2025





02/10/2025



2. Evidencia de Resultados

Evidencia de resultados de la entrega (detallados en la rúbrica de evaluación).
Para esta entrega tuvimos dos actividades principales:

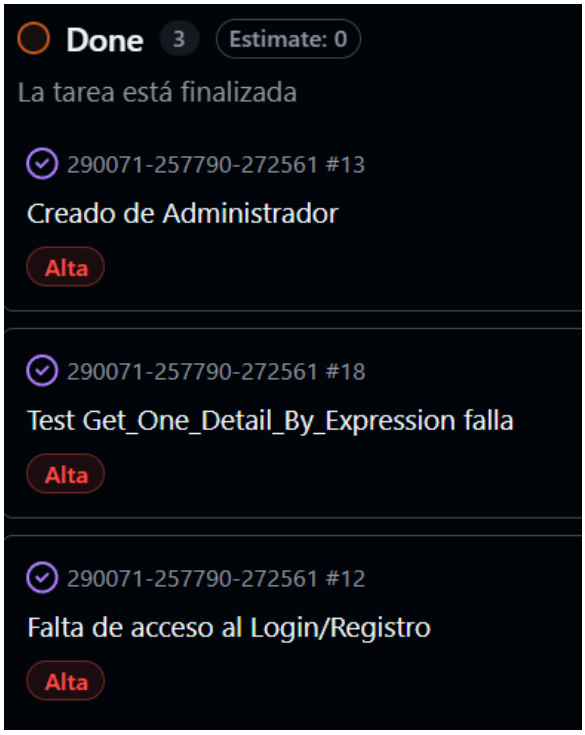
- Configuración del pipeline CI/CD
- Elección y realización de dos issues del backlog.

En el repo, en /Informes/segunda_entrega.md se encuentra la descripción del Pipeline realizado con cada uno de los pasos definidos e implementados.
También, en el mismo archivo, se encuentran los Issues seleccionados y la justificación de su selección.

Nota: En el markdown se aclara se seleccionaron 2 Issues, pero también se realizó un tercero para poder aplicar y probar el Pipeline implementado, ya que si no lo solucionamos, la sección de test fallaba.

Link al repositorio creado en la organización:
[IngSoft-ISA2-2025-2/290071-257790-272561]

A continuación, captura de pantalla de los issues solucionados:



También agregamos captura de pantalla de la página con los nuevos botones de login y register que se pueden visualizar a la derecha arriba:



Por último captura de pantalla de parte del nuevo código implementado en un nuevo servicio, que se ejecuta al iniciar el programa y controla que el admin exista, de lo contrario lo crea:

```
public void Initialize()
{
    EnsureRolesExist();
    EnsureAdminUserExists();
}
```

3. Referencias y Documentación

Link directo a la documentación MarkDown donde se encuentra el informe realizado en base a la rúbrica:

290071-257790-272561/Informes/segunda entrega.md at main · IngSoft-ISA2-2025-2/290071-257790-272561

4. Mejoras y Revisión con el PO (Review)

Mejoras en la calidad y evidencia de revisión con el PO (uno de los integrantes debe tomar el rol en cada etapa).
Durante esta etapa, estos fueron los roles realizados (implementando rotación del PO respecto la instancia anterior):
- Ivan: Developer, Tester, Product Owner
- Matías: Dev, Tester
- Francisco: Dev, Tester, Scrum Master

Análisis del PO:

- El pipeline CI/CD funciona en las PR a main (build, tests y coverage).
- La app ahora crea el usuario admin al iniciar, evitando el bloqueo de “primera ejecución”.
- Se agregaron botones de Login y Registro en la página principal, mejorando el acceso.
- Se arregló el test Get_One_Detail_By_Expression para estabilizar la CI.
- Falta avanzar en seguridad (hash de contraseñas, JWT con expiración/refresh, Bearer, HTTPS, validación de entrada).
    - A nivel código, el servicio de inicialización está ordenado y aislado. Quedan mensajes de error genéricos por mejorar.

Cosas que se mejoraron:

- Accesibilidad y funcionalidad de login/registro con botones visibles.
- Primer uso o despliegue de la aplicación sin problemas: creación automática del admin.
- Estabilidad del pipeline: tests en verde y cobertura aceptable..
- Trazabilidad: issues ↔ PR ↔ tablero Kanban y documento /Informes/segunda_entrega.md actualizado.

Resultados de la Review:
El incremento fue aprobado y se integró a main. Los PRs quedaron mergeados y los issues, cerrados. El pipeline corrió en verde y se confirmó que la cobertura mínima para permitir el merge.

Además, en base al análisis de seguridad, se concretó de solucionar problemas respecto a dicho atributo ya que son de gran importancia en el proyecto.

Conclusiones Review
La revisión cerró de forma positiva: el pipeline CI/CD demostró ser confiable (build, tests y coverage en verde), se resolvieron problemas clave de uso inicial (creación automática del admin) y se mejoró la entrada a la app con botones visibles de login/registro.

Además, se estabilizó un test que bloqueaba la integración. Quedó claro que el siguiente foco debe ser Seguridad y Mantenibilidad: priorizar hashing de contraseñas, JWT con expiración/refresh y Bearer, forzar HTTPS y sumar validaciones de entrada, junto con ordenar mensajes de error y nombres de métodos.

Con esto, la base para futuras entregas es más sólida y el proceso de integración está bajo control.

5. Retrospectiva y Acciones de Mejora (Retrospective)

Retrospectiva y acciones de mejora

Aspectos positivos:

- El pipeline dio feedback rápido y evitó integrar cambios rotos.
- Buena trazabilidad entre issues, PRs y tablero.
- Comunicación fluida (Discord) y Swarming ante bloqueos.
- Se respetó el WIP, evitando sobrecarga.

Aspectos negativos:

- Coordinar horarios costó y demoró el inicio de algunas tareas.
- TDD irregular para cobertura: Si bien se aplicó TDD, iniciando con los test básicos, un test se ajustó después del código para lograr la cobertura deseada.
- No se avanzó en el proyecto en el aspecto de seguridad a pesar de tener prioridad alta.
- Falta plantilla de PR con checklist, las descripciones variaron.

Cosas a mejorar:

- Crear plantilla de PR con checklist (DoR/DoD, evidencias, link a issue).
- Reforzar TDD: guía breve y casos límite por bug antes de cada PR.
- Planificar y ejecutar la épica “Seguridad Base” (hash, JWT + expiración/refresh, Bearer, HTTPS, validación/sanitización).

Conclusiones Retrospective

El equipo mostró buen flujo de trabajo: trazabilidad clara entre issues y PRs, comunicación efectiva y respeto del WIP.

Sin embargo, hay mejoras necesarias para sostener la calidad: coordinar horarios con una franja fija, aplicar TDD de forma más pareja (evitando ajustes de último momento) y ejecutar de verdad las tareas de seguridad que ya están priorizadas.

Con una plantilla de PR + checklist, guías breves de TDD y una planificación concreta de la épica “Seguridad Base”, la próxima iteración debería llegar con menos defectos y un incremento más seguro.

Parte 2

1. Configuración del pipeline CI/CD

A continuación haremos una descripción de los múltiples pasos del Pipeline de forma ordenada.

1. Desarrollo(Manual)
   En esta primera etapa, el desarrollador trabaja de manera local en su entorno configurado. Aquí se corrigen bugs reportados o se implementan nuevas funcionalidades siguiendo la metodología TDD (Test Driven Development).

   Para mantener un control adecuado sobre el flujo de cambios, cada nueva tarea o issue se desarrolla en una rama independiente. Esto asegura que ningún cambio en progreso llegue a la rama main sin haber pasado previamente por las validaciones correspondientes.
   Este enfoque favorece la trazabilidad del código, el control de versiones y la detección temprana de errores.

2. Commit (Manual)
   Durante el desarrollo, el programador realiza commits frecuentes en su rama local, respetando las convenciones de nombres definidas por el equipo. Estas convenciones permiten identificar rápidamente el tipo de cambio realizado:

- **Green**: se utiliza cuando se implementa la solución mínima necesaria para que los tests pasen exitosamente.
- **Refactor**: se utiliza cuando se realizan mejoras al código (ya sea en estilo, estructura o eficiencia) sin alterar el comportamiento funcional ni romper los tests.

   Esto asegura un historial de commits claro, ordenado y fácil de seguir durante la revisión.

3. Creación Pull Request(Manual)
   Una vez finalizada la implementación de la solución, el desarrollador abre una Pull Request (PR) hacia la rama main.
   Cada PR debe contener:

- Descripción detallada del problema resuelto.
- Referencia al issue vinculado, asegurando la trazabilidad entre el problema reportado y la solución.
- Evidencia de la ejecución de los tests, como capturas de pantalla o reportes generados.

   El objetivo es proporcionar suficiente contexto para que los revisores comprendan el cambio y lo evalúen adecuadamente.

4. Build (Github Action - Automático)

   Al momento de crear la PR, se activa automáticamente un workflow en GitHub Actions. Este proceso automatizado realiza las siguientes tareas:

   1.1. Descarga el código del repositorio.

   1.2. Compila el proyecto, asegurando que el código fuente sea válido.

   1.3. Verifica las dependencias instaladas y detecta posibles errores de compilación.

   De esta forma, se establece una primera capa de validación técnica antes de que el código pueda ser revisado por otros integrantes del equipo.

5. Unit test (Github Action - Automático)

   Si la compilación fue exitosa, el workflow continúa con la ejecución de todos los tests unitarios definidos en el proyecto.
   El objetivo principal de esta etapa es:

- Confirmar que los cambios introducidos no rompen funcionalidades existentes.
- Validar que la implementación cumple con las reglas de TDD, es decir, que los tests fueron diseñados y aprobados antes de la integración final.

   En caso de fallar algún test, la PR no puede avanzar y debe ser corregida por el desarrollador.

6. Coverage (Github Action - Automatico)

   A continuación, se ejecuta un análisis de cobertura de código. Esta etapa mide qué porcentaje del código está siendo evaluado por los tests automatizados.
   El reporte generado queda asociado directamente a la PR y permite:

- Validar que las áreas críticas del sistema se encuentran debidamente cubiertas.
- Asegurar que se cumple con el porcentaje mínimo de cobertura definido por el equipo.

   Si la cobertura es insuficiente, el merge no podrá ser realizado hasta que se refuercen los tests.

7. Peer review PR (Manual)

   Superadas las validaciones automáticas, la Pull Request pasa a revisión manual por parte del equipo.
   Los reviewers analizan distintos aspectos de la solución:

- Calidad del código: claridad, legibilidad y respeto por las convenciones.
- Impacto en la arquitectura: verificación de que no se introduzcan problemas de diseño.

- Cobertura y calidad de tests: confirmación de que los escenarios críticos estén cubiertos.
- Resolución del issue: se valida que el problema reportado quede efectivamente solucionado.

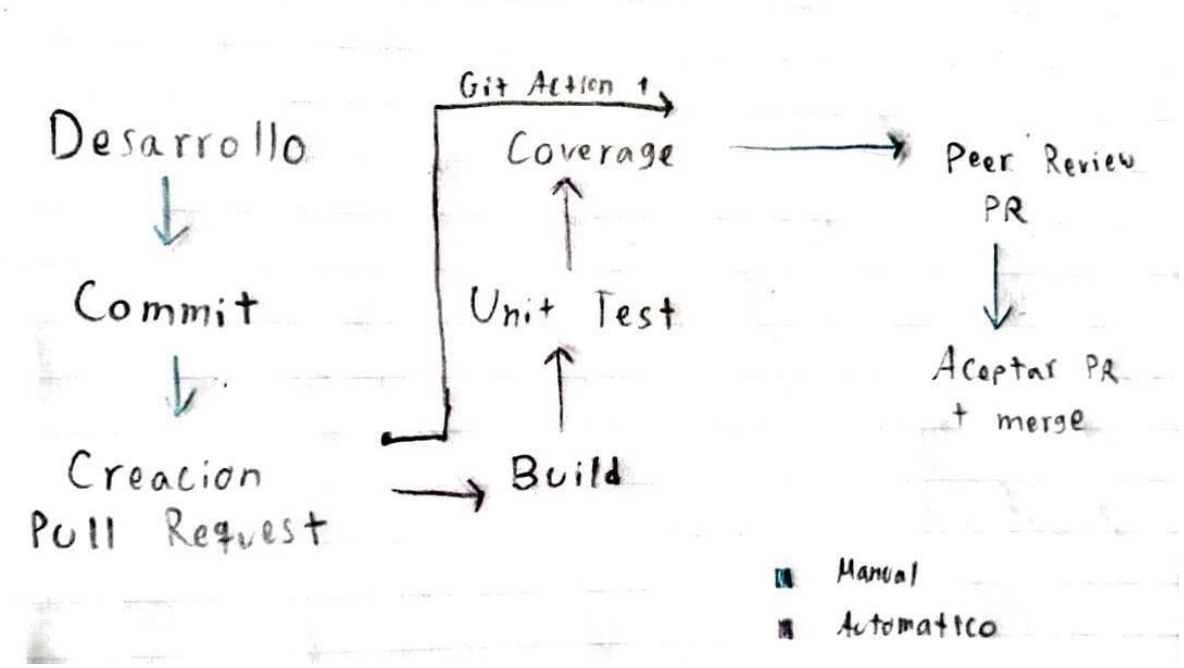
  En esta etapa se pueden sugerir mejoras o solicitar cambios adicionales antes de aprobar la integración.

8. Aceptar PR + merge (Manual)

   Cuando la Pull Request cumple con todos los requisitos, un reviewer autorizado aprueba la integración y procede a realizar el merge a la rama main.
   Al completar este paso se realizan tareas complementarias para mantener el proyecto organizado:

- Se etiqueta la versión resultante con el nombre "Entrega 2".
- Se actualiza el tablero Kanban, reflejando el avance del flujo de trabajo.
- Se cierra el issue correspondiente, asegurando la trazabilidad del ciclo de desarrollo.

  Este proceso garantiza que cada entrega pase por validaciones automáticas y manuales antes de llegar a la rama principal.



2. Elección y realización de Issues

1. Creado de administrador

Descripción del problema: Para que la aplicación pueda ser utilizada correctamente, necesita tener un usuario administrador creado, el cual no se crea automáticamente.

Justificación de selección en base al modelo de calidad:
Decidimos empezar por este problema, ya que es necesario solucionarlo para ejecutar la aplicación por primera vez.
Este problema corresponde al atributo de Funcionalidad dentro del modelo de calidad, porque sin la creación automática del usuario administrador el sistema no puede usarse correctamente desde el inicio. Es decir, falta una función esencial para que la aplicación cumpla con lo esperado y sea operativa desde la primera ejecución.

Solución: Cada vez que la aplicación se inicia, se controla que exista un usuario admin, de lo contrario, se crea uno en la base de datos.

2. Falta de acceso al login/registro

Descripción del problema: Para que la página sea funcional, y pueda llevar un registro de quien compro que, es necesario tener la capacidad de iniciar sesión (o registrarse de ser necesario) a la hora de hacer la aplicación. Dicha funcionalidad ya está implementada, pero hasta ahora para acceder a la misma había que modificar la url de la página manualmente, sin existir un botón para acceder directamente.

Justificación de selección en base al modelo de calidad:
Decidimos arreglar este problema debido a que es fundamental a la hora de acceder a una página web, el hecho de poder logearse/registrarse en ella. Creemos que no es necesario el hecho de que el login/registro sea la primera ventana que se ve al abrir la página debido a la existencia de varias paginas (como mercado libre o amazon) en la que se puede visualizar directamente los productos antes de registrarte.

Solución: En la ventana principal se puede ver agregado los botones de logueo y de registro respectivamente, uno a lado del otro. Estos botones tienen asociados la página correspondiente.

Nota

Si bien no lo consideramos como parte de las issues seleccionadas, tuvimos que realizar la siguiente issue debido a los parámetros configurados en las Github Actions:

- Test Get_One_Detail_By_Expression falla

El test Get_One_Detail_By_Expression fallaba, por lo que tuvimos que arreglar el test para que las pull requests puedan ser aceptadas por el pipeline y mantener la consistencia de que todos los test deben pasar para poder realizar la integración en main.

Entrega 3

Parte 1

Creado de User Stories

1. Crear la reserva de un medicamento

Título: Crear la reserva de un medicamento
Narrativa:

Como usuario
quiero poder crear una reserva de medicamento
para asegurarme que tenga lo que necesito asegurado

Pregunta 1: ¿Qué es una reserva? Una forma de asegurarme de que exista un medicamento cuando llegue al local a retirarlo y pagarlo
Pregunta 2: ¿Quien la puede hacer? Un cliente (“usuario anónimo”)
Pregunta 3: ¿Qué información debe ingresar el cliente? Tipo de medicamento y cantidad del medicamento

Regla 1: Tiene que haber stock del medicamento para reservarse
Regla 2: El stock ingresado por el cliente debe ser mayor a 0
Regla 3: El usuario debe ingresar un medicamento para reservar
Regla 4: El usuario tiene que ingresar una cantidad para reservar
Regla 5: El usuario ingresa a la página de reservar desde home

Ejemplo 1 (regla 1): Usuario que intenta reservar y hay Stock
Given un usuario que está en la url “/reservarMedicamento”
And estoy logueado como "Usuario Anónimo"
When ingreso el medicamento “Ibuprofeno” y la cantidad “10”
And hay stock de “Ibuprofeno”
And hago click en el botón “Reservar”
Then se reserva el medicamento con estado "Pendiente"
Ejemplo 2 (regla 1): Usuario que intenta reservar y no hay Stock
Given un usuario que está en la url “/reservarMedicamento”
When ingreso el medicamento “Ibuprofeno” y la cantidad “10”
And no hay stock de “Ibuprofeno”
And hago click en el botón “Reservar”
Then se muestra el mensaje de error "No hay stock disponible"

Ejemplo 1 (regla 2): Usuario intenta crear reserva con cantidad igual o menor a 0
Given un usuario intenta reservar un medicamento pidiendo una cantidad igual o menor a 0

When intenta crear la reserva
Then el sistema no se lo permite
Ejemplo 2 (regla 2): Usuario intenta crear reserva con cantidad mayor a 0
Given un usuario intenta reservar un medicamento pidiendo una cantidad mayor a 0
When intenta crear la reserva
Then el sistema lo permite

Ejemplo 1 (regla 3): Reservar sin ingresar un medicamento
Given un usuario que está en la url “/reservarMedicamento”
And estoy logueado como "Usuario Anónimo"
When ingreso la cantidad “10” sin medicamento
And hago click en el botón “Reservar”
Then se muestra el mensaje de error "Debe ingresar un medicamento"
Ejemplo 2 (regla 3): Reservar ingresando un medicamento
Given un usuario que está en la url “/reservarMedicamento”
And estoy logueado como "Usuario Anónimo"
When ingreso el medicamento “Ibuprofeno” y la cantidad “10”
And hago click en el botón “Reservar”
Then se reserva el medicamento

Ejemplo 1 (regla 4): Reservar sin ingresar una cantidad
Given un usuario que está en la url “/reservarMedicamento”
And estoy logueado como "Usuario Anónimo"
When ingreso el medicamento “Ibuprofeno” sin cantidad
And hago click en el botón “Reservar”
Then se muestra el mensaje de error "Debe ingresar una cantidad"
Ejemplo 2 (regla 4): Reservar ingresando una cantidad
Given un usuario que está en la url “/reservarMedicamento”
And estoy logueado como "Usuario Anónimo"
When ingreso el medicamento “Ibuprofeno” y la cantidad “10”
And hago click en el botón “Reservar”
Then se reserva el medicamento

Ejemplo 1 (regla 5): Ingresar a la página de reserva
Given un usuario que está en la url “/home”
And estoy logueado como "Usuario Anónimo"
When hago click en el botón “Reservar”
Then se redirige a la pagina “/reservarMedicamento”
Ejemplo 2 (regla 5): Volver a la página de home
Given un usuario que está en la url “/reservarMedicamento”
And estoy logueado como "Usuario Anónimo"
When hago click en el botón “Atrás”
Then se redirige a la página “/home”

2. Gestionar reservas de un medicamento

Titulo:

Como usuario
quiero poder ver las reservas que hice
para saber si me falto algo o tengo todo

Pregunta 1: ¿Que significa “gestionar” una reserva? Significa poder visualizar en una lista las reservas, donde estará la información de cada reserva y un botón de cancelar.
Pregunta 2: ¿Que información tiene una reserva? Tipo de medicamento, cantidad de medicamento, fecha de expiración de la reserva

Regla 1: El usuario puede ver una lista con la información de las reservas y un botón de cancelar

Ejemplo 1 (regla 1): Un usuario quiere gestionar sus reservas
Given un usuario que quiere gestionar sus reservas
When hace click en el botón “gestionar reservas”
Then es direccionado a la página para gestionar sus reservas

3. Cancelar una reserva de medicamentos (IVAN)

Titulo:

Como usuario
quiero poder cancelar las reservas
para sacar los medicamentos que no quiero de la lista

Pregunta 1: Que implica “cancelar una reserva”? Implica que al el cliente no querer el producto, el sistema lo elimina de la lista de reservados

Regla 1: No se puede cancelar una reserva ya confirmada

Ejemplo 1 (regla 1): Cliente quiere cancelar un producto reservado, pero esta confirmado
Given un cliente que quiere cancelar un producto reservado
When lo quiere cancelar
Then no le aparece la opción, al ya estar “Confirmado”
Ejemplo 2 (regla 1): Cliente quiere cancelar un producto reservado, pero NO esta confirmado
Given un cliente que quiere cancelar un producto reservado
When lo quiere cancelar
Then le permite cancelarlo, al no estar “Confirmado”

4. (Empleado) Confirmar reserva medicamentos (FRAN)

Título:

Como empleado
quiero poder confirmar las reservas de los usuarios
para verificar que todos los requisitos se cumplan

Pregunta 1: Hay algo más que confirmar además de la prescripción? No

Regla 1: Si tiene prescripción, debe tener los pre requisitos para ser confirmado
Regla 2: Si no tiene prescripción, el empleado confirma sin más

Ejemplo 1 (regla 1): Empleado confirma reserva al haber prescripción del medicamento
Given Un empleado con una reserva con prescripción
When lo va a confirmar
Then lo confirma al tener la prescripción necesaria para el medicamento
Ejemplo 2 (regla 1): Empleado no confirma reserva al no haber prescripción del medicamento
Given Un empleado con una reserva con prescripción
When lo va a confirmar
Then no lo confirma al no tener la prescripción necesaria para el medicamento

Ejemplo 1 (regla 2): Empleado confirma reserva al no necesitar prescripción del medicamento
Given Un empleado con una reserva sin prescripción
When lo va a confirmar
Then lo confirma al no necesitar prescripción

Aclaraciones:
1. Tuvimos que cambiar la estructura de las carpetas y los archivos cypress.config.ts, package.json y tsconfig.json para que funcionaran las pruebas de cypess.

2. El escenario 2 de crear reserva podría no funcionar en otro idioma

3. Los test de cypress de crear reserva solo funcionan si existe una droga con nombre “Droga1” y stock 200.

Diagrama de componentes

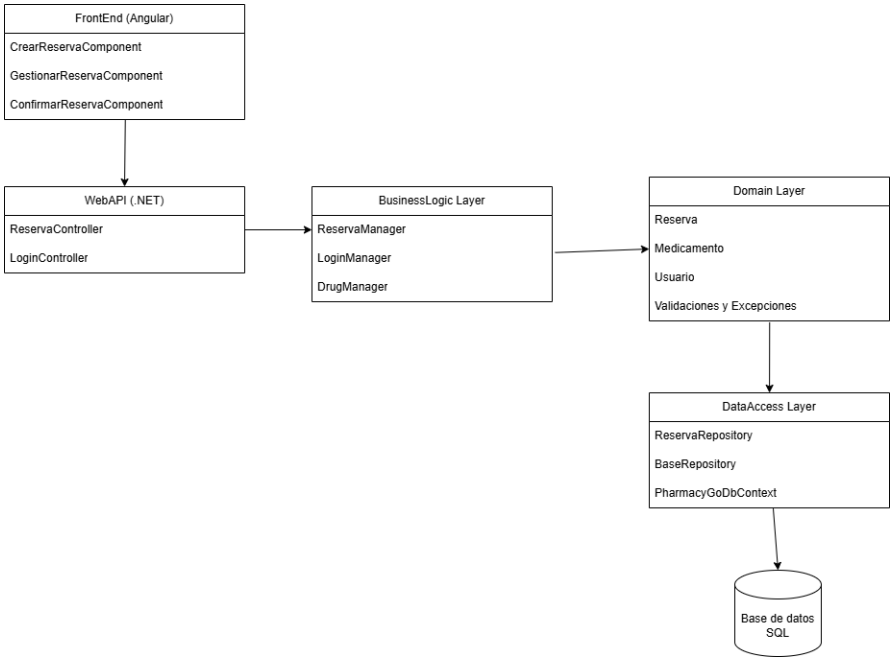


Diagrama de Secuencia Gestión de Reserva:

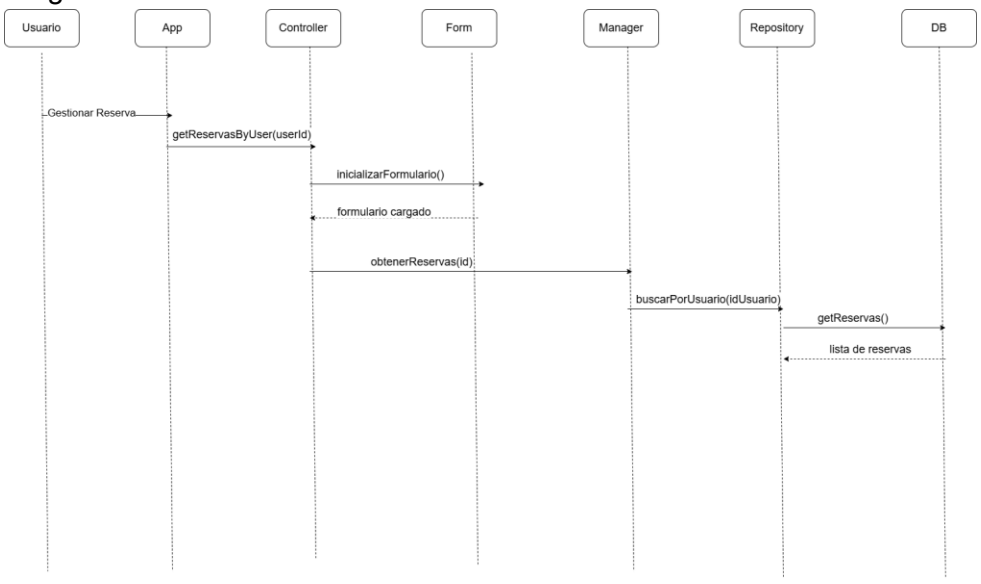
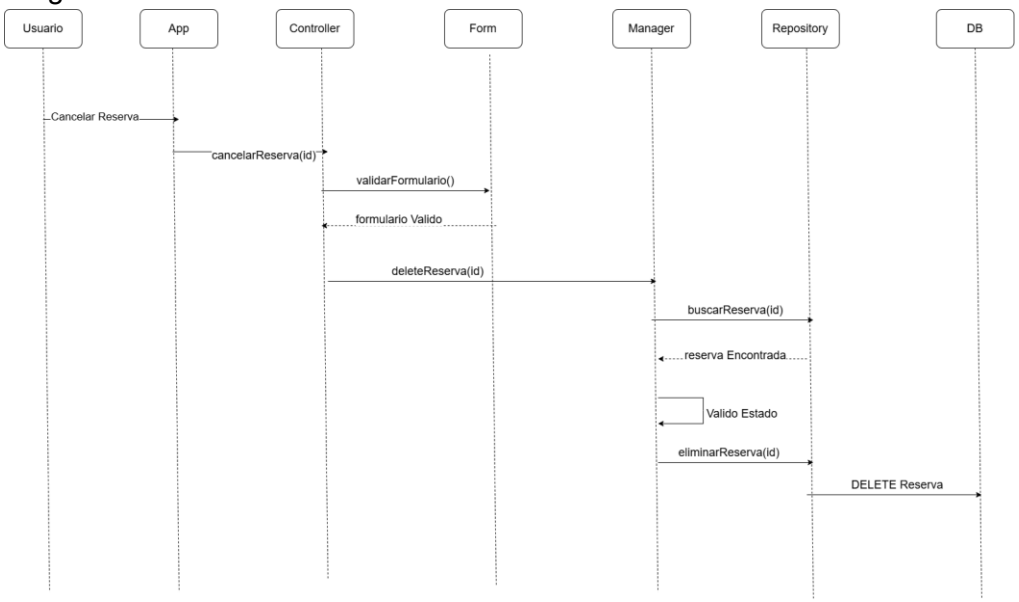


Diagrama de secuencia Cancelación Reserva:



Parte 2

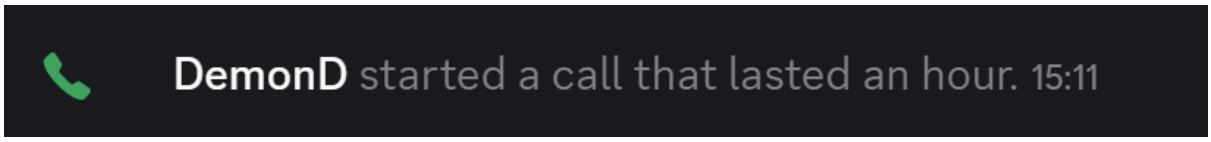
1. Registro de actividad:

11/10/25



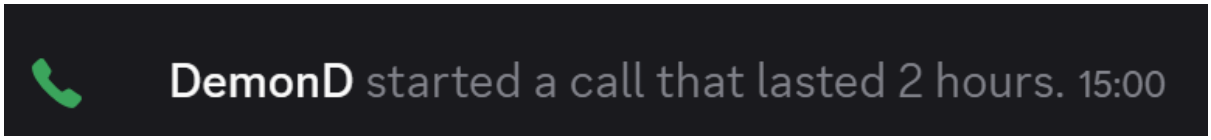
Empezamos a hacer las US. Falta refinar algunas de ellas.

15/10/25



Agregado de Cypress y arreglo en estructuras de las User Stories

21/10/25





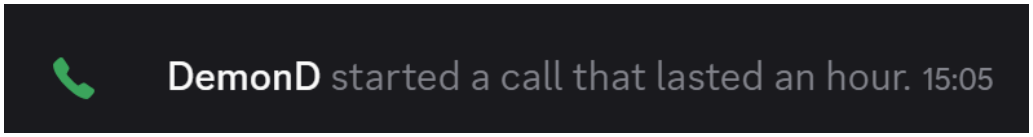
Empezamos con BDD RED (por primera vez) de crear reserva

22/10/25



Realizamos BDD RED, TDD RED (por primera vez) de crear reserva

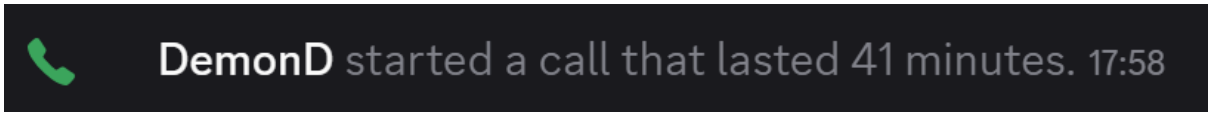
23/10/25

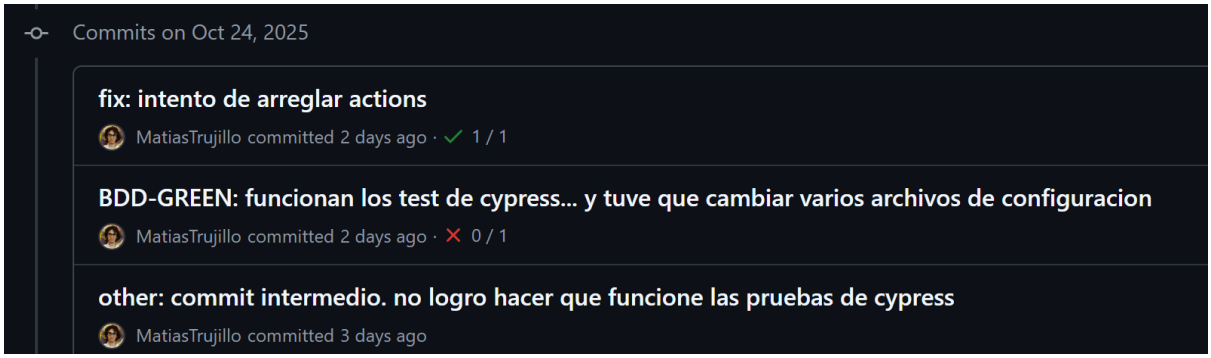




Separación de tareas.
Realizamos TDD GREEN, BDD GREEN (por primera vez) de crear reserva

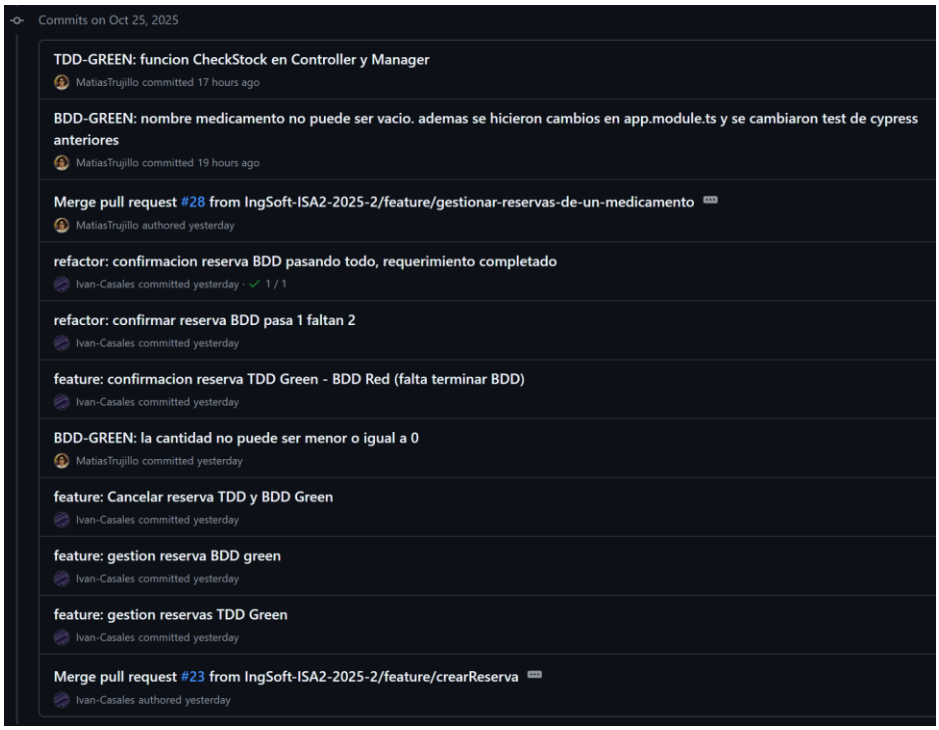
24/10/25





Solución de problema cypress, fin de primer ciclo BDD.

25/10/25



Implementamos todas las US.

26/10/25
Escribimos la documentación y terminamos los últimos detalles

2. Evidencia de resultados

Para esta entrega tuvimos tres actividades principales:

- Creación de US con el formato BDD.
- Implementación de las US con BDD.
- Creación de diagramas de secuencia y componentes.

Capturas sobre los commits se encuentran en la sección de arriba de “Registro de actividades”.

Capturas sobre algunos tests de Cypress:



3. Referencias y documentación

Link directo a la documentación MarkDown donde se encuentra el informe realizado en base a la rúbrica:

290071-257790-272561/Informes/tercera_entrega.md at main · IngSoft-ISA2-2025-2/290071-257790-272561

4. Mejoras y Revisión con el PO (Review)

Durante esta etapa, estos fueron los roles realizados (implementando rotación del PO respecto la instancia anterior):
- Ivan: Developer, Tester

- Matías: Dev, Tester, Product Owner
- Francisco: Dev, Tester, Scrum Master

Análisis del PO:

- Se agregaron paginas para crear reservas, gestionar y confirmar reserva.
- Se agregó un botón para acceder a crear reservas.
- No se agregó un botón para acceder a gestionar reservas.
- (boton para confirmar)
- La funcionalidad de gestionar reservas está incompleta, ya que no se puede ver las reservas por usuario, sino todas juntas.
- No se agregó la funcionalidad de seguridad con claves público-privadas.

Conclusión Review:
Se cumplió una parte importante del alcance, quedando pendientes algunas funcionalidades clave para su finalización.

5. Retrospectiva y Acciones de Mejora (Retrospective)

Retrospectiva y acciones de mejora
Aspectos positivos:

- Implementar BDD sirvió para crear US más útiles.
- La implementación de pruebas de front resultó útil y aceleró los tiempos de testing.

Aspectos negativos:

- Coordinar horarios costó y demoró el inicio de algunas tareas.
- Por falta de tiempo y por ser la primera vez en usar BDD, no se aplicó BDD de forma totalmente correcta. Por ejemplo, sólo hubo una instancia de crear US, no hubo un ciclo.

Cosas a mejorar:

- Realizar más ciclos BDD.
- Aprender a realizar mejores pruebas Cypress.

Conclusión de retrospective:
La implementación de BDD en el proyecto muestra potencial, aunque aún es necesario familiarizarse con la metodología para poder aprovechar plenamente.

Entrega 4

Parte 1

1. Implementar nuevo tipo de testing

Con el objetivo de fortalecer la seguridad del código fuente del proyecto, se incorporó un nuevo tipo de prueba: análisis estático de seguridad (SAST) mediante la herramienta Semgrep. Este tipo de testing analiza el código sin ejecutarlo, buscando patrones potencialmente vulnerables según reglas predefinidas.

Semgrep es una herramienta de análisis estático moderna que combina análisis semántico (AST) con búsqueda basada en patrones. Su propósito es detectar constructos de código que puedan derivar en vulnerabilidades de seguridad o malas prácticas de desarrollo.

- Ventajas principales:
    - Rápido y ligero
    - Lenguaje-consciente
    - Personalizable
    - Integrable en CI/CD
- Limitaciones:
    - No detecta vulnerabilidades que solo aparecen en tiempo de ejecución (por ejemplo, race conditions o errores de configuración dinámica).
    - Algunas reglas genéricas pueden generar falsos positivos si no se ajustan adecuadamente.
    - No reemplaza pruebas dinámicas (DAST) ni auditorías manuales, sino que las complementa.

Evidencia de ejecución

Al ejecutar Semgrep sobre el código del proyecto, el resultado del análisis (archivo SARIF generado, semg rep-results) reportó múltiples hallazgos asociados a la regla hardcoded-password, que identifica contraseñas o credenciales escritas directamente en el código fuente.

Ejemplos concretos:

En el archivo DatabaseInitializationService.cs se detectó la línea:

```
Password = "Admin123!"; // En producción, esto debería estar hasheado
```

→ Riesgo: exposición de credenciales en texto plano.

Estos hallazgos son relevantes ya que el uso de credenciales hardcodeadas puede permitir el acceso no autorizado, comprometer datos sensibles o generar malas prácticas replicadas en entornos productivos.

La incorporación de Semgrep como herramienta SAST permitió introducir un nuevo tipo de testing orientado a la seguridad del código fuente, alineado con las prácticas modernas de desarrollo seguro.
El análisis detectó vulnerabilidades reales (contraseñas hardcodeadas) y permitió definir mejoras para fortalecer la gestión de credenciales, cumpliendo con los objetivos de calidad del proyecto y aportando una capa adicional de prevención de riesgos de seguridad antes del despliegue.

2. Cálculo, justificación e interpretación de métricas DevOps

Change Lead Time:
Esta métrica mide el tiempo que tarda un commit en llegar a producción. Debido a que no tenemos despliegues en este proyecto, tomaremos el tiempo que tarda una Github Action en correr (las cuales se ejecutan cuando se hace un pull request).

Haciendo un promedio de cuanto tardaron las Github Actions de los merge correctos tenemos un Change Lead Time de: (77+131+86+84+81+79+90+91+92+85+96) / 11 = 90 segundos.

Esto nos dice que nuestro Change Lead Time es rápido, lo que logra una entrega rápida de valor (o en este caso, una entrega rápida de cambios a nuestro equipo).

En nuestro caso, aunque existen algunos valores algo más altos (como los 131 segundos), la mayoría de los tiempos se concentran alrededor del promedio (entre 79 y 96 s), lo que implica que el proceso está bajo control y no presenta variaciones significativas.

Change Fail Percentage:
Esta métrica mide el porcentaje de despliegues fallidos. Una vez más, como no tenemos despliegues, decidimos adaptar esta métrica a nuestro proyecto. Decidimos que contaremos las Github Actions fallidas como despliegues fallidos.

En este caso, observamos que nuestro Change Fail Percentage es de: 6/17 = 0,35 = 35%.

Este valor indica un porcentaje alto de errores. Sin embargo, la gran mayoría de estos errores (5 de los 6 errores) fueron al principio del proyecto, cuando aún estábamos configurando las Github Actions y familiarizándonos con el proyecto.

3. Implementación de telemetría

Métricas de negocio

- Ventas mensuales: cantidad total de ventas registradas por mes.
    - Permite evaluar el desempeño comercial general del sistema.
- Reservas mensuales: cantidad de reservas realizadas por mes.
    - Ayuda a medir la demanda y el uso real del servicio.

Métricas de aplicación

- Tiempo de respuesta de reserva: mide cuánto tarda la aplicación en responder cuando el usuario realiza una reserva (desde que el usuario hace click en "Reservar" hasta que se confirma la reserva).
    - Indica la rapidez y eficiencia percibida por el usuario.
- Tasa de errores: porcentaje de solicitudes de reserva que fallan.
    - Permite detectar fallas o inestabilidad en el sistema.

Métricas de infraestructura

- Uso promedio de CPU del contenedor: porcentaje de uso del procesador en el contenedor Docker.
    - Ayuda a identificar sobrecargas o necesidad de optimizar recursos.
- Uso promedio de memoria del contenedor: cantidad de memoria utilizada por el contenedor Docker.
    - Permite controlar el consumo y prevenir posibles fallas por falta de recursos.

Parte 2

Universidad ORT Uruguay
Facultad de Ingeniería

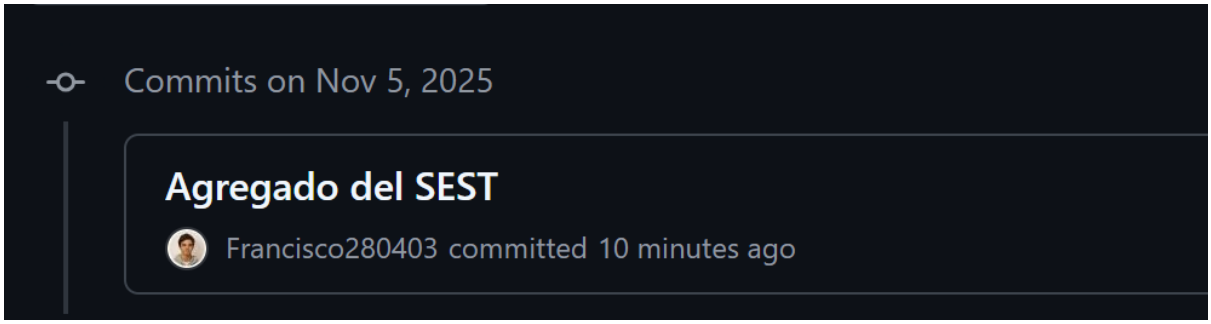


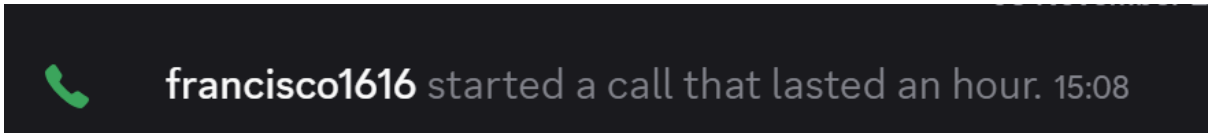
Entregable 4 - Ingeniería de Software Ágil 2

Estudiantes: Iván Casales (257790), Francisco Gandioli (290071), Matías Trujillo (272561)
Profesores: Santiago Gonzalez, Juan Irabedra
Año de entrega: 2025

1. Registro de actividad

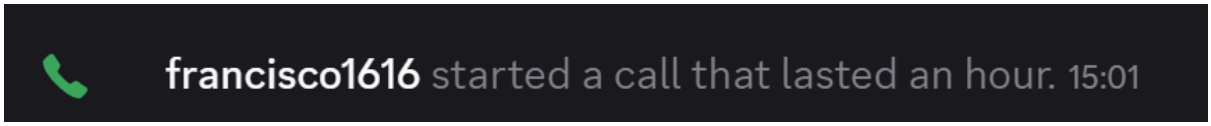
5/11/25

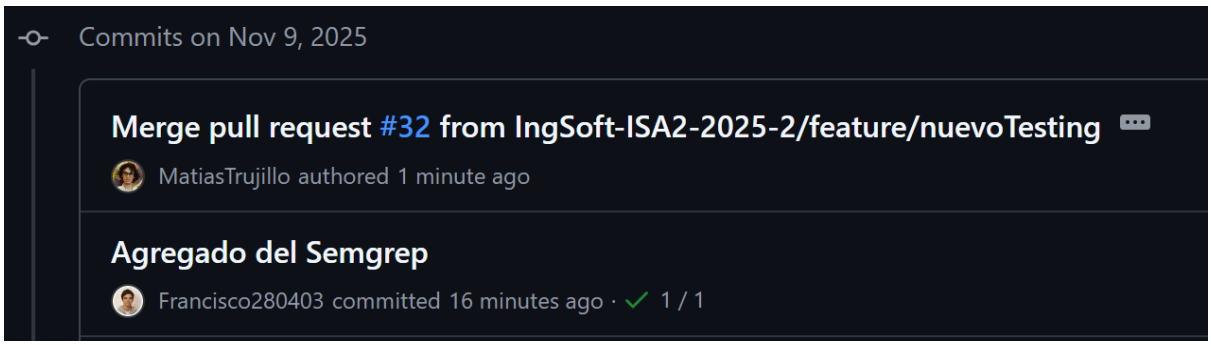




Implementamos semgrep en el proyecto.

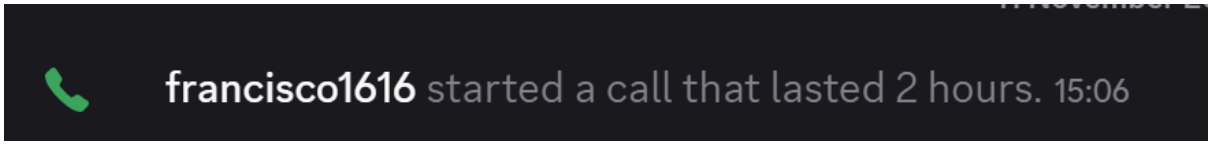
9/11/25





Definimos las métricas e hicimos los cálculos.

11/11/25



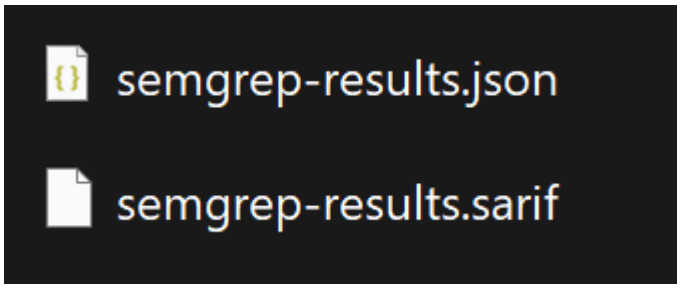
2. Evidencia de resultados

Para esta entrega tuvimos tres actividades principales:

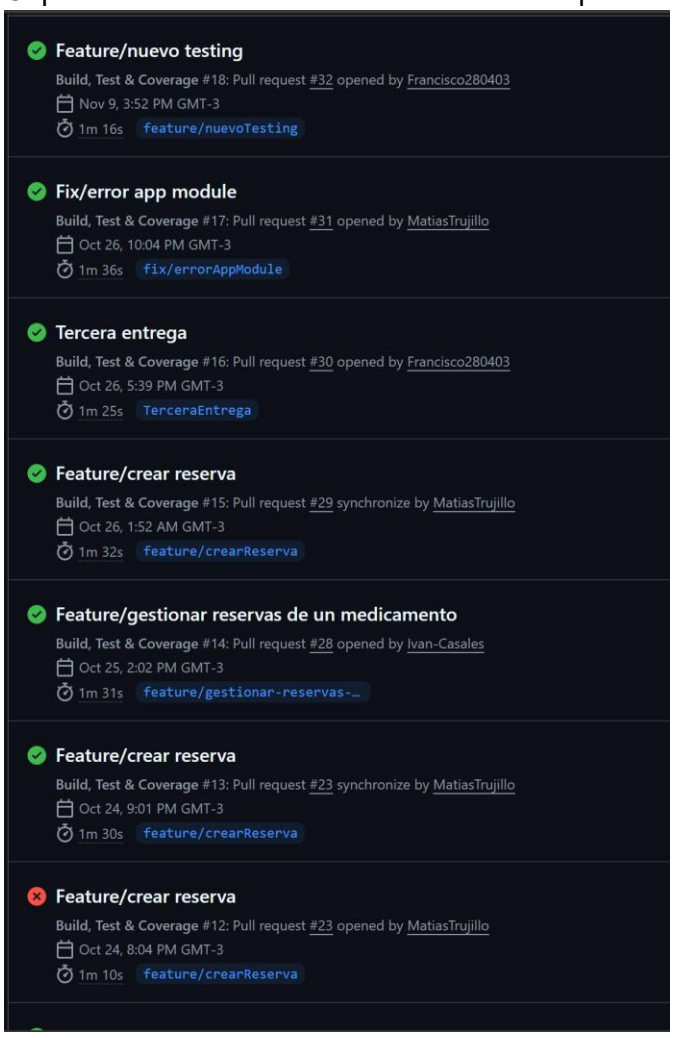
- Implementación de un nuevo tipo de testing

- Cálculo de métricas DevOps
- Implementación de telemetría

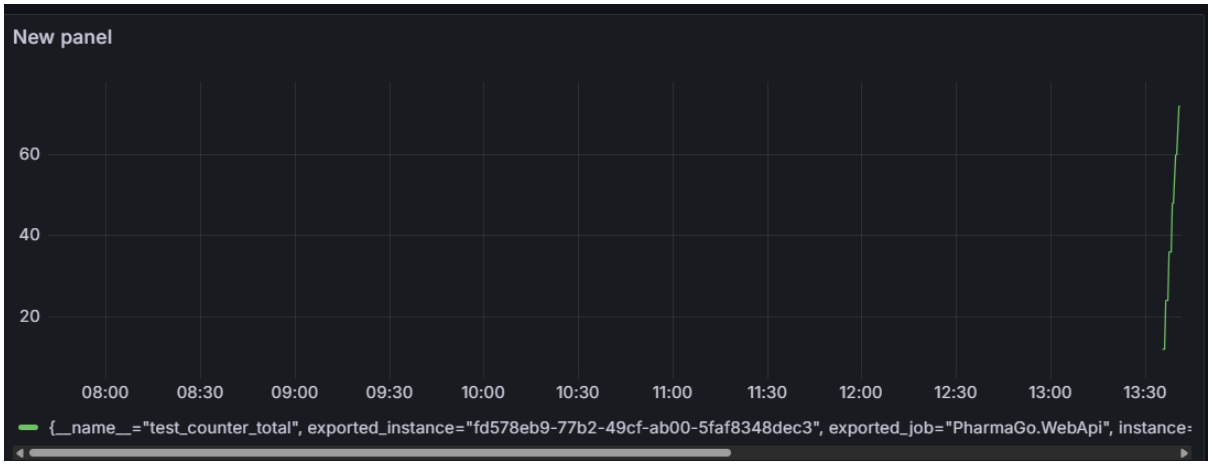
Capturas sobre el nuevo tipo de testing:



Capturas de dónde sacamos la información para calcular las métricas:



Captura de las métricas:



Si bien las métricas son independientes entre ellas, al hacer las consultas al mismo tiempo, y todas responder al mismo ecento/oleada de trafico, las gráficas quedan practicamente iguales unas a otras. No obstante, cada metrica representa una dimension distinta, y la foto Es por esto que es lo mismo subir todas las fotos de las métricas, o simplemente una.

Se adjunta un archivo metrics.md en la carpeta docs en el repo, en donde se explica cómo levantar y consultar las pruebas de las métricas.

3. Referencias y documentación

Link directo al Repositorio de Github:
IngSoft-ISA2-2025-2/290071-257790-272561

Link directo a la documentación MarkDown donde se encuentra el informe realizado en base a la rúbrica:
290071-257790-272561/Informes/4_cuarta_entrega.md at main · IngSoft-ISA2-2025-2/290071-257790-272561

4. Mejoras y Revisión con el PO (Review)

Durante esta etapa, estos fueron los roles realizados (implementando rotación del PO respecto la instancia anterior):
- Ivan: Developer, Tester, Scrum Master
- Matías: Dev, Tester, Product Owner
- Francisco: Dev, Tester

Análisis del PO:

- Se agregó un nuevo tipo de testing (SAST con Semgrep), que permitió encontrar vulnerabilidades que antes pasaban desapercibidas.
- La aplicación ahora envía métricas que pueden verse en Grafana, lo que facilita entender mejor cómo está funcionando el sistema.

Cosas que se mejoraron:

- El nuevo tipo de testing ayudó a detectar fallos de seguridad que no teníamos en cuenta.
- Las métricas permiten ver problemas o comportamientos extraños con más rapidez.

Resultados de la Review:

El incremento fue aprobado sin problemas. Los Pull Requests se mergearon correctamente, el pipeline se ejecutó en verde y se verificó que la cobertura mínima para permitir el merge se cumpliera.

Conclusiones Review

La revisión terminó de forma positiva. El SAST fue útil para encontrar errores reales, y las métricas mostradas en Grafana aportan más información para seguir mejorando el sistema.

Esto permitirá que el equipo tenga una mejor retroalimentación en próximas iteraciones.

5. Retrospectiva y Acciones de Mejora (Retrospective)

Aspectos positivos

- La comunicación del equipo fue buena y ayudó a avanzar sin mayores bloqueos.
- El análisis de métricas DevOps sirvió para entender el rendimiento del proyecto desde un punto de vista más técnico y medible.

Aspectos negativos

- Fue difícil coordinar horarios en algunos momentos.
- Algunos commits no respetaron la estructura que el equipo había acordado.

Acciones de mejora

- Crear un recordatorio visible con el formato correcto para los mensajes de commit, para que todos puedan seguirlo fácilmente.
- Intentar organizar mejor los horarios, por ejemplo planificando reuniones con un poco más de anticipación.

Estas mejoras son simples y ayudan a mantener un buen orden de trabajo para las próximas entregas.